

AIQT CLI

Milestone 2 Build Specification v0.3

Project Context Capture

Build-planning document for implementing AIQT Milestone 2. Not a runtime document model for AIQT user projects.

Field	Value
Product	AIQT CLI
Specification	Milestone 2 Build Specification
Version	v0.3
Milestone	M2 - Project Context Capture
Aligns With	AIQT MVP Specification v0.6 and Technical Architecture Specification v0.3
Depends On	M1 - Canonical State Foundation, commit 79798b6, tag m1-canonical-state-foundation
Primary Agent Target	Claude Code or equivalent coding agent
Implementation Status	Ready for build handoff

Document status: Ready for final review before build handoff

Contents

1. Source Alignment
 2. Milestone Objective
 3. Commands in Scope
 4. Out of Scope
 5. Behavioral Position in the Workflow
 6. Update Input Modes
 7. Input Precedence and Flag Mapping
 8. Update Patch Contract
 9. Record Identity, Merge, and Idempotency Rules
 10. Canonical Mutation Rules
 11. Planning Readiness Rule
 12. Runlog Rules
 13. Command Behavior
 14. Repository Implementation Scope
 15. Runtime and Package Impact
 16. Data and Schema Contracts
 17. Timestamp Rules
 18. Error Handling and Exit Codes
 19. Testing Scope
 20. Implementation Tasks
 21. Definition of Done
 22. Agent Execution Packet
- Appendix A - Minimum Test Matrix
- Appendix B - Example Update Input File

1. Source Alignment

This document is derived from AIQT MVP Specification v0.6 and AIQT Technical Architecture Specification v0.3. It assumes Milestone 1 is complete on main at commit 79798b6 and that the repository already contains the canonical state foundation, CommandResult contract, schemas, state stores, runlog store, deterministic JSON output, and validation tooling.

The governing design remains:

```
state -> graph -> packet -> checkpoint
not:
spec -> milestone file -> task file -> review file
```

Milestone 2 extends the state engine by allowing the user to capture durable project context. It must not start planning, generate a work graph, create milestones, generate agent packets, or export markdown documents.

2. Milestone Objective

Milestone 2 implements project context capture through `aiqt update`. It turns an initialized but mostly empty AIQT project into a project state that contains enough durable context for a later planning engine to create a work graph.

Milestone 2 must prove that AIQT can:

- run `aiqt update` against an existing `.aiqt/` project;
- capture or ingest durable project context;
- update `.aiqt/project.json` without corrupting existing state;
- append deterministic runlog events;
- keep `.aiqt/state.json` synchronized with the next recommended workflow action;
- return stable human-readable and JSON output;
- support non-interactive updates for testing and agent-driven workflows;
- support lightweight interactive intake for human project owners;
- leave the work graph empty until `aiqt plan` exists in a later milestone.

Milestone 2 is intentionally narrow. It implements context mutation, not planning.

3. Commands in Scope

```
aiqt update
aiqt update --json
aiqt update --from-file <path>
aiqt update --from-file <path> --json
aiqt update --objective "<objective>"
aiqt update --target-user "<target user>"
aiqt update --agent "claude-code"
aiqt update --repository-path "."
```

Direct flags are limited to stable project-level fields. Requirements, decisions, assumptions, risks, and open questions are provided through interactive intake or `--from-file` to avoid creating a large granular command surface.

--target-user may be repeatable. If the CLI framework implementation does not support repeatable flags initially, one target user value is acceptable for M2, but tests must cover at least one value.

4. Out of Scope

- aiqt plan implementation;
- work graph generation;
- milestone generation;
- work-unit generation;
- dependency generation;
- agent handoff packet generation;
- checkpoint capture;
- review service implementation;
- markdown exports;
- AIQT.md, AGENTS.md, CLAUDE.md, docs/, .milestones/, or .tasks/;
- delete, remove, or archive operations for project records;
- complex conflict resolution or migrations;
- Claude Code API integration;
- GitHub integration;
- autonomous execution.

5. Behavioral Position in the Workflow

M1: init/status/next -> M2: update -> M3: plan -> M4: next agent packet -> M5: checkpoint

After M1, aiqt next recommends aiqt update because project context is incomplete. After M2 captures sufficient context, aiqt next should recommend aiqt plan. aiqt plan does not need to exist yet, but the recommendation must be correct so the workflow has a clear next action.

6. Update Input Modes

6.1 Direct flag mode

Direct flags update project-level fields only. They are intended for CI, smoke tests, and quick human updates.

Flag	Canonical field	Behavior
--objective	project.objective	Sets the objective if the value is non-empty.
--target-user	project.targetUsers	Appends then deduplicates target users. Repeatable where supported.
--agent	project.preferredAgent	Sets the preferred coding agent if non-empty.
--repository-path	project.existingRepositoryPath	Sets the existing repository path if non-empty.
--from-file	structured update input	Loads a JSON patch before direct flags are applied.

6.2 --from-file mode

--from-file accepts a JSON patch matching UpdateInputSchema. Omitted fields mean no change. Empty strings must not overwrite existing values. Invalid JSON or schema-invalid input must return exit code 3.

6.3 Interactive mode

When aiqt update is executed without flags and the process has a TTY, M2 must offer a lightweight interactive intake. This is not a full planning interview.

Interactive mode is entered only when no direct update flags and no --from-file argument are supplied. Therefore, aiqt update --from-file patch.json is non-interactive file mode; aiqt update --objective "X" is non-interactive flag mode; and aiqt update --from-file patch.json --objective "X" is non-interactive file-plus-flag mode where flags override file values.

Interactive prompts must appear in this order:

1. Project objective
2. Target users
3. Preferred agent
4. Existing repository path
5. Constraints
6. Non-goals
7. Technology preferences
8. Business rules
9. Architecture notes
10. Requirements
11. Decisions
12. Assumptions
13. Risks
14. Open questions
15. Preferred validation commands

Interactive rules:

- Empty answers are allowed and mean no change.
- Existing scalar values should be shown as defaults where practical.
- String-array prompts may accept comma-separated values.
- Object-array prompts may support zero or more records using a small add-more loop.

- For object records, prompt only for the fields listed in the input schemas and apply defaults.
- Interactive mode must not create a work graph or ask the user to approve a plan.
- If prompt tooling is unavailable in the current process, return `requiresHumanInput = true` and exit code 10 rather than guessing.

6.4 Non-TTY no-input behavior

If `aiqt update` is executed without flags in a non-interactive environment, it must return `requiresHumanInput = true` and exit code 10. It must not mutate any file.

7. Input Precedence and Flag Mapping

When `--from-file` and direct flags are supplied together, M2 must use deterministic merge order:

1. Load and validate the `--from-file` patch first.
2. Apply direct flags second.
3. For scalar project fields, direct flags override file values.
4. For `targetUsers`, direct flags append and deduplicate because M2 has no destructive replace semantics.
5. Non-overlapping file values are preserved.
6. If the merged patch is a no-op, do not write files and do not append runlog events.

Example:

```
aiqt update --from-file patch.json --objective "New Objective"
```

Result:

```
- project.objective = "New Objective"
- targetUsers from file remain unless --target-user is also supplied
- all other valid patch.json values still apply
```

8. Update Patch Contract

The `--from-file` input is a JSON object representing a partial update. It is not stored directly as a canonical file. The service validates the patch, merges it into `project.json`, and writes canonical state.

```
{
  "project": {
    "objective": "Build a local CLI workflow engine for AI agent implementation control.",
    "targetUsers": ["Human project owner using AI coding agents"],
    "preferredAgent": "claude-code",
    "existingRepositoryPath": "."
  },
  "context": {
    "constraints": ["Local file-based state only"],
    "nonGoals": ["No hosted backend in MVP"],
    "technologyPreferences": ["Node.js", "TypeScript", "pnpm"],
    "businessRules": ["Markdown is generated only on request"],
    "architectureNotes": ["Source of truth is structured state"]
  },
  "requirements": [
    {
      "clientKey": "req-init-state",
```

```

    "title": "Initialize project state",
    "description": "The CLI can initialize canonical .aiqt files.",
    "priority": "critical",
    "type": "functional",
    "acceptanceCriteria": ["aiqt init creates only canonical files"],
    "status": "accepted"
  }
],
"decisions": [
  {
    "clientKey": "decision-state-source-of-truth",
    "decision": "Use structured state as source of truth.",
    "reason": "Prevents document sprawl and supports resumable workflows.",
    "impact": "Markdown is only an export format.",
    "status": "decided"
  }
]
}

```

9. Record Identity, Merge, and Idempotency Rules

Object arrays use deterministic identity resolution. This section is authoritative for requirements, decisions, assumptions, risks, and openQuestions.

9.1 Input identity fields

Each object input may include:

- `id?: string` - used to update an existing canonical record by stable AIQT ID;
- `clientKey?: string` - user-provided stable key used to make repeated `--from-file` updates idempotent.

`clientKey` is optional but recommended for `--from-file` repeatability. `clientKey` must be persisted on the canonical record when supplied.

9.2 Merge order for object arrays

1. If `id` is supplied and matches an existing record of the same type, update that record.
2. Else if `clientKey` is supplied and matches an existing record of the same type, update that record.
3. Else create a new record with a generated stable ID.
4. If `id` is supplied but does not match an existing record of the same type, reject the update with `status = failed`, `exitCode = 3`, a critical blocking issue in area = input, and no mutation. Import/migration exceptions are out of scope for M2.
5. If neither `id` nor `clientKey` is supplied, every invocation creates a new record.

9.3 Idempotency

Re-running the same `--from-file` patch with the same `clientKey` values must not create duplicate records. If the effective merged canonical state is unchanged, M2 must perform a no-op: no timestamp bump, no file write, and no runlog event.

9.4 Field merge behavior

- Missing fields mean no change when updating an existing record.
- Empty strings are ignored and must not overwrite existing values.
- For simple string arrays, use append-and-deduplicate semantics.
- For acceptanceCriteria and preferredValidationCommands, append and deduplicate.
- Null is accepted only for nullable fields and means intentionally set the field to null.
- Deleting existing records, deleting array items, or clearing non-nullable fields is out of scope.

10. Canonical Mutation Rules

- project.json is the primary file mutated by aiqt update.
- state.json may only be updated for nextRecommendedCommand and lastUpdatedAt unless a deterministic status correction is required.
- state.projectStatus must remain draft until aiqt plan generates a work graph. M2 must not set projectStatus to planned.
- state.workGraph.milestones, state.workGraph.workUnits, and state.workGraph.dependencies must remain unchanged by M2.
- runlog.jsonl receives append-only events for successful mutations.
- No human-readable markdown files are created by aiqt update.
- No removal semantics are implemented in M2. Record deletion is out of scope.
- IDs are stable after creation and must not be regenerated.
- Canonical JSON field names remain camelCase.
- AIQT_SCHEMA_VERSION should remain 0.5.0. Optional clientKey support and timestamping of M2-created records are backward-compatible with empty M1 projects.

11. Planning Readiness Rule

M2 must include a small deterministic readiness helper that decides the next recommended command. This is not the planning engine.

Planning context is sufficient when:

1. project.objective is non-empty;
2. project.targetUsers has at least one item;
3. at least one of the following exists: one accepted or draft requirement, one context constraint, one technology preference, one business rule, or one architecture note;
4. no openQuestion has impact = "blocking" and status = "open".

If sufficient, update state.nextRecommendedCommand to aiqt plan. If insufficient, keep or set state.nextRecommendedCommand to aiqt update. state.projectStatus remains draft in both cases until M3 implements aiqt plan.

12. Runlog Rules

- Append project.updated once for each successful update command that changes project.json or state.json.
- Append decision.recorded once for each newly created decision.
- decision.recorded must not be emitted for unchanged or updated decisions.
- Events must use the existing RunlogEventSchema from M1.
- Event IDs must use the stable ID generator and continue from the highest existing EVT id.
- Event actor is human for interactive, direct flag, and --from-file input in M2.
- relatedIds must include PROJECT-001 and any created or modified record IDs.
- A no-op update must not append a runlog event.

12.1 project.updated event data shape

```
{
  "id": "EVT-002",
  "type": "project.updated",
  "timestamp": "ISO_DATE",
  "actor": "human",
  "summary": "Project context updated.",
  "relatedIds": ["PROJECT-001", "REQ-001", "D001"],
  "data": {
    "changedFields": ["project.objective", "project.targetUsers", "requirements", "decisions"],
    "changedSections": ["project", "requirements", "decisions"],
    "createdRecordIds": ["REQ-001", "D001"],
    "updatedRecordIds": [],
    "planningContextReady": true,
    "nextRecommendedCommand": "aiqt plan"
  }
}
```

12.2 decision.recorded event data shape

```
{
  "id": "EVT-003",
  "type": "decision.recorded",
  "timestamp": "ISO_DATE",
  "actor": "human",
  "summary": "Decision recorded.",
  "relatedIds": ["PROJECT-001", "D001"],
  "data": {
    "decisionId": "D001",
    "decision": "Use structured state as source of truth.",
    "reason": "Prevents document sprawl and supports resumable workflows.",
    "impact": "Markdown is only an export format."
  }
}
```

13. Command Behavior

13.1 aiqt update

Purpose: capture durable project context for a project already initialized by aiqt init.

Responsibilities:

1. Locate .aiqt/.

2. Read project.json and state.json.
3. Validate canonical schemas and version compatibility.
4. Collect update input from interactive prompts, --from-file, direct flags, or combined file-plus-flags mode.
5. Validate the merged update patch.
6. Apply canonical mutation rules.
7. Generate stable IDs for new records.
8. Apply record identity and idempotency rules.
9. Update project.updatedAt if project.json changes.
10. Update state.nextRecommendedCommand using the planning readiness rule.
11. Update state.lastUpdatedAt if state.json changes.
12. Append runlog events if mutation occurs.
13. Return a CommandResult.

13.2 aiqt update --json

The JSON output must conform to the existing CommandResult contract and expose deterministic data fields.

```
{
  "status": "passed",
  "action": "update",
  "projectStatus": "draft",
  "currentMilestoneId": null,
  "currentWorkUnitId": null,
  "summary": "Project context updated.",
  "completedActions": ["Read project.json", "Updated project context", "Appended runlog events"],
  "changedFiles": [".aiqt/project.json", ".aiqt/state.json", ".aiqt/runlog.jsonl"],
  "affectedItems": ["PROJECT-001", "REQ-001", "D001"],
  "blockingIssues": [],
  "warnings": [],
  "requiresHumanInput": false,
  "nextRecommendedCommand": "aiqt plan",
  "data": {
    "changedFields": ["project.objective", "project.targetUsers", "requirements", "decisions"],
    "changedSections": ["project", "requirements", "decisions"],
    "createdRecordIds": ["REQ-001", "D001"],
    "updatedRecordIds": [],
    "planningContextReady": true,
    "noOp": false
  },
  "exitCode": 0
}
```

13.3 aiqt next after M2

M2 may need to refine the existing next-action helper so aiqt next recommends aiqt plan after context is sufficient and no work graph exists. aiqt next must still not mutate state in M2.

14. Repository Implementation Scope

Recommended files to add or modify:

```
src/
  cli/
    commands/
      update.command.ts
    prompts/
      update-prompts.ts
  core/
    schema/
      update-input.schema.ts
    workflow/
      planning-readiness.ts
      next-action.ts          # may be updated
  services/
    project-update-service.ts
  core/state/
    project-store.ts          # may be extended only if needed
    workflow-state-store.ts   # may be extended only if needed
    runlog-store.ts          # may be extended only if needed
    ids.ts                   # may be extended only if needed
  tests/
    unit/
      update-input.schema.test.ts
      project-update-service.test.ts
      planning-readiness.test.ts
    integration/
      update.command.test.ts
      update-next-flow.test.ts
    fixtures/
      update-input/
        valid-minimal.json
        valid-full.json
        idempotent-client-key.json
        invalid-schema.json
```

Do not create planning-service.ts, work-graph-service.ts, agent-packet-service.ts, checkpoint-service.ts, review-service.ts, or export-service.ts in M2 unless they already exist as empty placeholders from M1. M2 should not implement their behavior.

15. Runtime and Package Impact

- Continue using Node.js, TypeScript, ESM, pnpm, commander, zod, and vitest.
- Use @inquirer/prompts only if it is not already installed and interactive intake requires it.
- Do not add dependencies for planning, markdown processing, graph traversal, external APIs, or GitHub.
- package.json.version may remain unchanged unless the implementation process intentionally bumps package metadata.
- AIQT_SCHEMA_VERSION must remain 0.5.0 unless canonical file schema changes incompatibly. M2 should not require a schema version bump.

16. Data and Schema Contracts

16.1 UpdateInputSchema

The update input schema is a patch schema. It validates proposed changes before mutation. It is not stored directly as a canonical file.

Strictness rule: UpdateInputSchema and all nested supported input objects must reject unknown keys. This includes project, context, quality, RequirementInputSchema, DecisionInputSchema, AssumptionInputSchema, RiskInputSchema, and OpenQuestionInputSchema. Unknown keys at any supported input level must fail with exit code 3.

```
export const ProjectUpdateInputSchema = z.object({
  objective: z.string().min(1).optional(),
  targetUsers: z.array(z.string().min(1)).optional(),
  preferredAgent: z.string().min(1).nullable().optional(),
  existingRepositoryPath: z.string().min(1).nullable().optional(),
}).strict();

export const ContextUpdateInputSchema = z.object({
  constraints: z.array(z.string().min(1)).optional(),
  nonGoals: z.array(z.string().min(1)).optional(),
  technologyPreferences: z.array(z.string().min(1)).optional(),
  businessRules: z.array(z.string().min(1)).optional(),
  architectureNotes: z.array(z.string().min(1)).optional(),
}).strict();

export const QualityUpdateInputSchema = z.object({
  acceptanceCriteriaRequired: z.boolean().optional(),
  validationRequiredBeforeDone: z.boolean().optional(),
  preferredValidationCommands: z.array(z.string().min(1)).optional(),
}).strict();

export const UpdateInputSchema = z.object({
  project: ProjectUpdateInputSchema.optional(),
  context: ContextUpdateInputSchema.optional(),
  requirements: z.array(RequirementInputSchema.strict()).optional(),
  decisions: z.array(DecisionInputSchema.strict()).optional(),
  assumptions: z.array(AssumptionInputSchema.strict()).optional(),
  risks: z.array(RiskInputSchema.strict()).optional(),
  openQuestions: z.array(OpenQuestionInputSchema.strict()).optional(),
  quality: QualityUpdateInputSchema.optional(),
}).strict();
```

16.2 Record input schemas

Record	Identity fields	Minimum create fields	Defaults
RequirementInput	id?, clientKey?	title, description	priority=medium; type=functional; acceptanceCriteria=[]; status=draft
DecisionInput	id?, clientKey?	decision	reason=""; impact=""; status=decided; date=current YYYY-MM-DD
AssumptionInput	id?, clientKey?	statement	reason=null; source=human; status=active

RiskInput	id?, clientKey?	title, description	severity=medium; mitigation=null; status=open
OpenQuestionInput	id?, clientKey?	question	impact=medium; status=open; answer=null

For updates by id or clientKey, partial inputs are allowed. The update is valid if the final merged canonical record validates.

16.3 Canonical record timestamp and clientKey fields

All object records created by aiqt update should include createdAt and updatedAt. If clientKey is supplied, it should be stored on the canonical record as an optional field. Existing records without clientKey remain valid.

16.4 ID Prefixes

Record type	ID prefix	Examples
Requirement	REQ-	REQ-001, REQ-002
Decision	D	D001, D002
Assumption	ASM-	ASM-001, ASM-002
Risk	RISK-	RISK-001, RISK-002
Open Question	Q	Q001, Q002
Runlog Event	EVT-	EVT-002, EVT-003

17. Timestamp Rules

- project.updatedAt is updated whenever project.json is actually mutated.
- On record create: createdAt = current ISO date and updatedAt = current ISO date.
- On record update: createdAt is preserved and updatedAt = current ISO date.
- On no-op: do not update project.updatedAt, record updatedAt, state.lastUpdatedAt, or runlog.
- state.lastUpdatedAt is updated only when state.json changes.
- Use one timestamp value per command execution where practical to keep output deterministic within the command result.

18. Error Handling and Exit Codes

All errors must render through the existing CommandResult and Issue structures where possible. JSON mode must not emit terminal formatting.

Case	Exit code	Status	Mutation
.aiqt/ missing	3	failed	No
project.json or state.json invalid	3	failed	No
unsupported future schema version	3	failed	No
--from-file path missing or unreadable	3	failed	No
--from-file invalid JSON	3	failed	No
UpdateInputSchema validation fails	3	failed	No

unknown id supplied for update	3	failed	No
no input in non-TTY	10	needs_input	No
valid no-op update	0	passed or warning	No
successful mutation	0	passed	Yes

19. Testing Scope

- All existing M1 tests must continue to pass.
- New unit tests must cover UpdateInputSchema, merge semantics, ID creation, timestamp behavior, and planning readiness.
- New integration tests must cover aiqt update with direct flags, --from-file, combined file-plus-flags mode, and no-input non-TTY behavior.
- At least one flow test must confirm init -> update -> next recommends aiqt plan.
- Tests must confirm workGraph remains unchanged after update.
- Tests must confirm no markdown files are created.
- Tests must confirm runlog events are appended only on mutation.
- Tests must confirm repeated --from-file with clientKey is idempotent.
- Tests must confirm decision.recorded is emitted only for newly created decisions.
- Tests must confirm invalid inputs return deterministic exit codes and JSON shapes.

20. Implementation Tasks

Task	Scope	Acceptance
M2-T1 - CLI registration	Register update command and flags.	aiqt update appears in help; unknown flags fail deterministically.
M2-T2 - UpdateInputSchema	Add patch schema and record input schemas.	Valid minimal/full files pass; invalid schema fails exit 3.
M2-T3 - Planning readiness helper	Implement deterministic sufficient-context rule.	Unit tests cover ready/not-ready/blocking-question cases.
M2-T4 - Project update service	Merge patches into project.json and state.json.	Direct flags and file input mutate canonical files correctly.
M2-T5 - Identity and idempotency	Implement id/clientKey merge behavior.	Repeated clientKey patch does not create duplicates or runlog noise.
M2-T6 - Interactive prompts	Implement lightweight TTY prompts or clean no-input block.	Non-TTY no-input returns exit code 10; TTY path is available where testable.
M2-T7 - Runlog events	Append project.updated and decision.recorded.	Event IDs continue; event data shape is deterministic.
M2-T8 - next-action refinement	Update next helper for context-ready/no-graph state.	init -> update -> next recommends aiqt plan.
M2-T9 - Output rendering	Human and JSON outputs use CommandResult.	--json key order and data fields are stable.
M2-T10 - Tests and validation	Unit and integration coverage.	pnpm validate passes and M1 behavior is preserved.

21. Definition of Done

- aiqt update executes successfully against an initialized project.
- aiqt update --json emits deterministic CommandResult JSON.
- aiqt update --from-file ingests a valid update patch.
- direct project-level flags update objective, target users, preferred agent, and repository path.
- --from-file plus direct flags follow the defined precedence rules.
- interactive intake exists for TTY usage or is cleanly blocked with exit code 10 where TTY is unavailable.
- project.json is updated atomically and schema-valid after every successful update.
- state.json remains schema-valid and only updates nextRecommendedCommand and lastUpdatedAt as allowed.
- state.workGraph remains empty/unchanged in M2.
- records created by M2 have stable IDs, timestamps, and optional clientKey persistence.
- re-running the same clientKey-based --from-file patch is idempotent.
- runlog.jsonl receives valid append-only events for successful mutations.
- project.updated and decision.recorded event shapes match this specification.
- no markdown, milestone files, task files, or export files are created.
- aiqt next recommends aiqt plan after sufficient context is captured.
- all existing M1 tests still pass.
- new M2 unit and integration tests pass.
- pnpm typecheck, pnpm lint, pnpm test, pnpm build, and pnpm validate all pass.
- The project is ready for Milestone 3: aiqt plan / Dynamic Work Graph Generation.

22. Agent Execution Packet

```
# AGENT EXECUTION PACKET

## Role
You are implementing Milestone 2 of AIQT CLI.
AIQT is a local TypeScript CLI workflow state engine for projects built with AI coding agents.

## Current Work Unit
ID: AIQT-M2
Title: Project Context Capture
Objective: Implement aiqt update so an initialized AIQT project can capture durable project context
into .aiqt/project.json and recommend aiqt plan when enough context exists.

## Starting Point
M1 is complete on main.
Commit: 79798b6
Tag: m1-canonical-state-foundation

## Scope
Implement:
- aiqt update
- aiqt update --json
- aiqt update --from-file <path>
- aiqt update --from-file <path> --json
- aiqt update --objective "<objective>"
- aiqt update --target-user "<target user>"
- aiqt update --agent "claude-code"
- aiqt update --repository-path "."
```

- UpdateInputSchema
- project-update-service
- planning readiness helper
- id/clientKey record merge behavior
- idempotent --from-file behavior when clientKey is supplied
- interactive update prompts where TTY is available
- non-TTY no-input handling with exit code 10
- project.json mutation with atomic writes
- state.json nextRecommendedCommand update
- runlog append events
- deterministic human and JSON output
- unit and integration tests

Out of Scope

Do not implement:

- aiqt plan
- work graph generation
- milestone generation
- work-unit generation
- dependency generation
- agent packet generation
- checkpoint handling
- review service
- export service
- markdown document generation
- AIQT.md
- AGENTS.md
- CLAUDE.md
- GitHub integration
- Claude API integration

Constraints

- Keep canonical JSON camelCase.
- Do not bump AIQT_SCHEMA_VERSION unless strictly necessary.
- Do not mutate state.workGraph in M2.
- Do not create files outside the canonical .aiqt/ model.
- Use existing CommandResult, Issue, ExitCode, schemas, stores, and output renderers from M1.
- Preserve all existing M1 behavior.
- Apply file-plus-flag precedence exactly as specified.
- Use id/clientKey merge rules exactly as specified.

Acceptance Criteria

- pnpm typecheck passes.
- pnpm lint passes.
- pnpm test passes.
- pnpm build passes.
- pnpm validate passes.
- aiqt update can populate project context from --from-file.
- aiqt update can update objective, target user, agent, and repository path through direct flags.
- aiqt update --json returns deterministic CommandResult JSON.
- project.json remains schema-valid after update.
- state.json remains schema-valid after update.
- runlog.jsonl contains valid project.updated events after mutation.
- decision.recorded events are appended for new decisions only.
- repeated --from-file using clientKey does not create duplicate records.
- aiqt next recommends aiqt plan after sufficient context exists.
- no markdown/docs/milestone/task files are created.
- workGraph remains unchanged by update.

Validation Commands

pnpm typecheck


```

pnpm lint
pnpm test
pnpm build
pnpm validate

## Required Agent Output
Return:
1. Summary of implementation.
2. Files created.
3. Files modified.
4. Tests added.
5. Validation commands run.
6. Any unresolved issues.
7. Suggested next step.

```

Appendix A - Minimum Test Matrix

Test Type	Scenario	Expected Result
Unit	UpdateInputSchema validates valid minimal patch	Pass
Unit	UpdateInputSchema rejects invalid enum values	Exit code 3 through command path
Unit	UpdateInputSchema rejects unknown nested keys	Exit code 3 through command path
Unit	Planning readiness with objective, target user, and requirement	Ready -> aiqt plan
Unit	Planning readiness with blocking open question	Not ready -> aiqt update
Unit	ID generator creates REQ/D/ASM/RISK/Q IDs	Stable next ID
Unit	clientKey match updates existing record	No duplicate
Unit	unknown id in patch	Invalid input, exit code 3
Unit	timestamp create/update/no-op behavior	Correct timestamp mutation only
Integration	aiqt update --objective updates project.json	project.objective changed
Integration	aiqt update --target-user appends/dedupes	targetUsers stable
Integration	aiqt update --repository-path maps correctly	existingRepositoryPath set
Integration	aiqt update --from-file valid-full.json	project/context/records created without interactive prompts
Integration	repeat same --from-file with clientKey	No duplicate records, no-op on second run
Integration	--from-file plus --objective	flag objective wins
Integration	successful mutation appends project.updated	valid JSONL event
Integration	new decision appends decision.recorded	one event per new decision
Integration	updated decision does not append decision.recorded	no new decision event
Integration	no-op update	no file writes and no runlog event
Integration	non-TTY no-input update	requiresHumanInput true, exit code 10
Integration	init -> update -> next	nextRecommendedCommand = aiqt plan

Integration	workGraph after update	unchanged from before update
Integration	no markdown files created	AIQT.md/AGENTS.md/CLAUDE.md/ docs absent
Regression	all M1 tests	Still pass

Appendix B - Example Update Input File

```
{
  "project": {
    "objective": "Create a local CLI that manages AI-agent implementation workflow state.",
    "targetUsers": ["Human project owner using Claude Code"],
    "preferredAgent": "claude-code",
    "existingRepositoryPath": "."
  },
  "context": {
    "constraints": [
      "Local files are the source of truth",
      "No markdown files are created by default"
    ],
    "nonGoals": [
      "No SaaS backend in MVP",
      "No autonomous code execution in MVP"
    ],
    "technologyPreferences": ["Node.js", "TypeScript", "pnpm", "Vitest"],
    "businessRules": [
      "Human-readable exports are generated only on request"
    ],
    "architectureNotes": [
      "project.json stores durable context",
      "state.json stores workflow position and work graph"
    ]
  },
  "requirements": [
    {
      "clientKey": "req-capture-project-context",
      "title": "Capture project context",
      "description": "The update command stores durable project context in project.json.",
      "priority": "critical",
      "type": "functional",
      "acceptanceCriteria": [
        "aiqt update --from-file updates project.json",
        "aiqt update --json returns a deterministic CommandResult"
      ],
      "status": "accepted"
    }
  ],
  "decisions": [
    {
      "clientKey": "decision-single-update-command",
      "decision": "Use aiqt update as the single context mutation command.",
      "reason": "Avoids a large command surface with add-requirement/add-decision commands.",
      "impact": "Granular actions are handled internally by the update service.",
      "status": "decided"
    }
  ],
  "assumptions": [
    {
      "clientKey": "assumption-agent-external-mvp",
      "statement": "The implementation agent runs outside AIQT for MVP.",
    }
  ]
}
```

```

    "reason": "Deep provider integration is deferred.",
    "source": "human",
    "status": "active"
  }
],
"risks": [
  {
    "clientKey": "risk-overbuild-m2",
    "title": "Overbuilding the planning engine too early",
    "description": "M2 could accidentally drift into plan generation.",
    "severity": "medium",
    "mitigation": "Keep workGraph empty and defer aiqt plan.",
    "status": "open"
  }
],
"openQuestions": [
  {
    "clientKey": "question-m3-heuristic",
    "question": "Which planning heuristic should M3 use first?",
    "impact": "medium",
    "status": "open",
    "answer": null
  }
],
"quality": {
  "preferredValidationCommands": ["pnpm validate"]
}
}

```